

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA – FIAP

GUSTAVO LIMA MARTINS

**FRAMEWORK INTELIGENTE PARA MODELAGEM AUTOMATIZADA
DE AMEAÇAS EM ARQUITETURAS DE SOFTWARE UTILIZANDO
VISÃO COMPUTACIONAL, OCR, GRANDES MODELOS DE LINGUAGEM
E A METODOLOGIA STRIDE EM AMBIENTE MULTICLOUD
(AWS, AZURE E GOOGLE CLOUD PLATFORM)**

SÃO PAULO, 2026

SUMÁRIO

1. INTRODUÇÃO	1
2. METODOLOGIA	2
2.1. Fase 1: Curadoria, construção e anotação do <i>dataset</i>	2
2.1.1. Microtarefa 1.1 – Coleta estratégica do corpus visual	2
2.1.2. Microtarefa 1.2 – Anotação dos limites de confiança	2
2.1.3. Microtarefa 1.3 – Aplicação de aumento de dados	3
2.1.4. Microtarefa 1.4 – Exportação do <i>dataset</i>	3
2.2. Fase 2: Treinamento e validação da visão computacional.....	3
2.3. Fase 3: Extração hierárquica, OCR e engenharia espacial	4
2.3.1. Microtarefa 3.1 – Triagem de inferência.....	4
2.3.2. Microtarefa 3.2 – OCR por fatiamento de matriz.....	4
2.3.3. Microtarefa 3.3 – Associação do rótulo da zona	4
2.3.4. Microtarefa 3.4 – Cálculo de contenção geométrica	5
2.4. Fase 4: Motor de modelagem STRIDE e contramedidas via LLM..	5
2.5. Fase 5: Interface do usuário e orquestração	6
3. RESULTADOS E DISCUSSÃO	6
3.1.1. <i>UI do framework</i> personalizada com a identidade visual da FIAP	6
3.2. Métricas de treinamento do modelo YOLOv8.....	10
3.2.1. Rotulagem do <i>dataset</i>	10
3.2.2. Desempenho por classes	10
3.2.3. Desempenho por época.....	13
3.2.4. Metadados do modelo final	13
4. CONCLUSÃO	14
5. REFERÊNCIAS BIBLIOGRÁFICAS.....	16

LISTA DE FIGURAS

Figura 1 - <i>User interface</i> do STRIDE-AI com identidade visual inspirada na FIAP	6
Figura 2 - Parecer técnico STRIDE resultante do envio de um diagrama de arquitetura AWS....	7
Figura 3 - Diagrama de pipeline do <i>framework</i> STRIDE-AI desenvolvido como MVP	9
Figura 4 - Distribuição de rótulos por classe e mapa de calor das regiões com mais classes ..	10
Figura 5 - Gráficos de progresso do aprendizado do modelo em face das épocas de treino	11
Figura 6 - Matriz de confusão normalizada das <i>bounding boxes</i> globais por classe	12
Figura 7 - Gráficos de progresso do aprendizado do modelo em face das épocas de treino	13

LISTA DE TABELAS

Tabela 1 - Estratificação da designação das tecnologias por atribuição e módulo	8
Tabela 2 - Performance média de acuracidade interpretativa das classes nos diagramas	11

1. INTRODUÇÃO

Há poucos meses, cerca de 600 mil torcedores de um time de futebol, se depararam com um anúncio da contratação do notável jogador Neymar – antigo portador da camisa de número 10 da seleção brasileira – feito no site oficial do Coritiba Foot Ball Club (BRANCO, 2025). No entanto, tratava-se de um incidente de segurança cibernética, que perdurou por uma hora no ar causando constrangimentos entre a instituição esportiva, os torcedores e o jogador. Além deste episódio, em 2024 aconteceu o mega *hackeamento* da controversa plataforma de relacionamentos *Ashley Madison* – uma vez que o marketing da empresa fomentava a infidelidade de pessoas em relacionamentos monogâmicos – cujo desfecho foi o suicídio de um usuário que teve suas relações extraconjugais expostas na internet (AMERISE, 2024). Não obstante, recentemente a Vercel – plataforma de hospedagem de sites e aplicações *web* – confirmou um acesso não autorizado a sistemas internos e o subsequente vazamento do banco de dados, chaves de acesso, contas de funcionários e código-fonte da companhia (VERCEL, 2026).

Sabendo disso, urge a necessidade do emprego da modelagem de ameaças em arquitetura de software, com o intuito da mitigação expressiva do risco mediante tentativas de ataque cibernético, outrossim, segundo a Redação iMasters (2025), a GII Research projeta que o mercado global de segurança de aplicações deve crescer de US\$ 13,64 bi para US\$ 30,41 bi até 2030, consoante a isso, segundo o levantamento da Iron Mountain e FT Longitude, apenas em 2024, as organizações ao redor do mundo perderam cerca de US\$ 14 bi por falhas ligadas à integridade dos dados.

Portanto, o *framework* desenvolvido objetiva realizar a modelagem automatizada de ameaças em arquitetura de *software*, via visão computacional, OCR, engenharia espacial entre elementos, orquestração de grafo municiado com *LLM* e a metodologia STRIDE para avaliação de diagramas *multicloud*, para tanto, o protótipo desenvolvido para este trabalho foi disponibilizado em um repositório público no GitHub (MARTINS, 2026a), assim como, há um vídeo elucidativo sobre a aplicação com demonstrativo de uso apresentado em vídeo no Youtube (MARTINS, 2026b).

2. METODOLOGIA

O plano de ação consistiu em cinco etapas:

1. Curadoria, construção e anotação do *dataset*;
2. Treinamento e validação da visão computacional;
3. Extração hierárquica, OCR direcionado e engenharia espacial;
4. Motor de modelagem STRIDE e contramedidas via LLM;
5. Interface do usuário e orquestração.

2.1. Fase 1: Curadoria, construção e anotação do *dataset*

O objetivo desta fase era cumprir a exigência de construir e anotar um *dataset* para treinar o modelo supervisionado na identificação de componentes de arquitetura de software.

2.1.1. Microtarefa 1.1 – Coleta estratégica do corpus visual

Houve busca e *download* de 63 diagramas de arquitetura de software – sendo 20 da AWS, 23 da Azure e 20 da Google – nos seguintes sites oficiais: Azure Architecture Center (MICROSOFT, 2026), AWS Reference Architecture Diagrams (AMAZON WEB SERVICES, 2026) e Centro de arquitetura do GCP (GOOGLE CLOUD, 2026).

2.1.2. Microtarefa 1.2 – Anotação dos limites de confiança

Utilizando-se da Roboflow – plataforma online com recursos especializados para aplicações de visão computacional (ROBOFLOW, 2026) – foram rotulados os elementos com *bounding boxes*, com uma taxonomia de classes simplificada para adequar-se à metodologia STRIDE, conforme a descrição a seguir:

- a) *trust_boundary* – engloba a fronteira lógica de confiança (áreas do diagrama com maior aglomeração de elementos);
- b) *actor* – delimita os ícones que correspondem aos usuários;

- c) *compute* – delimita os ícones relativos aos servidores;
- d) *database_storage* – mapeia os ícones associados aos provedores de banco de dados;
- e) *api_gateway* – rastreia os gerenciadores de rotas identificáveis conforme o serviço;
- f) *network_security* – associa serviços direcionados à segurança de dados;
- g) *data_flow* – sobrepõe milimetricamente as setas e linhas de conexão para análise de tráfego.

2.1.3. Microtarefa 1.3 – Aplicação de aumento de dados

Após a rotulagem, configurou-se o Roboflow para multiplicar o *dataset* original aplicando brilho ou contraste (de 0% a 15%) e espelhamento horizontal (*Flip*). Tal técnica fomentou um aumento significativo no *corpus* visual, o qual progrediu de 63 para 439 (MARTINS, 2026c).

2.1.4. Microtarefa 1.4 – Exportação do *dataset*

Geração e *download* a versão final do *dataset* expandido no formato nativo "YOLOv8 PyTorch" para o diretório previamente estipulado no repositório local.

2.2. Fase 2: Treinamento e validação da visão computacional

Nesta fase, foi feito o treinamento do modelo supervisionado (YOLOv8 versão nano), com um limite de 100 *epochs* e tamanho de 1024 do *imgsz* – tamanho da imagem – para o modelo ter ampla capacidade de ler os diagramas de arquitetura.

2.3. Fase 3: Extração hierárquica, OCR e engenharia espacial

Esta fase traduz caixas coloridas em um JSON inteligente, usando a matemática de contenção para otimizar o processamento e gerar o contexto hierárquico necessário para a segurança.

2.3.1. Microtarefa 3.1 – Triagem de inferência

O modelo YOLOv8 Nano (*fine-tuned* neste projeto) detecta as 7 classes do diagrama. A inferência roda em RGB (a cor é sinal semântico forte em ícones *cloud: compute, database* e *storage* se distinguem primariamente pela cor) com *imgsz=1024* — o mesmo tamanho do treino — e filtro global de confiança de 25%, calibrado para cortar o ruído da classe *data_flow* (a mais instável do modelo). As detecções são separadas em duas listas: *trust_boundaries* e demais componentes.

2.3.2. Microtarefa 3.2 – OCR por fatiamento de matriz

Em vez de rodar OCR na imagem inteira, cada *trust_boundary* é recortada e só o recorte vai ao EasyOCR (menos ruído, menos custo). Recortes com menos de 200 pixels de altura são ampliados antes da leitura. O rótulo oficial da zona (ex.: "VPC", "Backend Systems") é isolado por proximidade vetorial ao canto superior-esquerdo do recorte — a convenção de diagramas AWS/Azure/GCP para nomear fronteiras.

2.3.3. Microtarefa 3.3 – Associação do rótulo da zona

O YOLO detecta o ícone, mas o nome do serviço fica fora do *bbox* – *bounding boxes* – (ao lado/abaixo). A solução reúne todas as leituras de OCR da imagem num pool de coordenadas absolutas e associa a cada componente a leitura mais próxima do seu centróide, com limiar de distância proporcional à diagonal da imagem (escala com a resolução) e confiança mínima de 0.3.

2.3.4. Microtarefa 3.4 – Cálculo de contenção geométrica

Contenção geométrica: o centróide de cada componente é testado contra os retângulos das zonas (ponto-em-retângulo); com zonas aninhadas, vence a de menor área (a mais específica).

- a) *data_flows*: cada seta detectada é ligada aos 2 nós conectáveis mais próximos dos cantos do seu *bbox*. As conexões são não-direcionadas — o modelo não distingue origem de destino, e o prompt instrui o LLM a avaliar os dois sentidos.
- b) *proximity_hints*: sinal complementar e deliberadamente mais fraco — pares de componentes fisicamente próximos dentro da mesma zona para os quais não foi detectada seta (KNN com teto e piso de distância como fração da diagonal, dedup contra os *data_flows*). Recupera relações que o detector de setas perdeu, sem tratá-las como comunicação confirmada.

O resultado é um JSON hierárquico: zonas nomeadas com seus componentes aninhados, componentes órfãos (*unassigned_components*), fluxos e *hints*.

2.4. Fase 4: Motor de modelagem STRIDE e contramedidas via LLM

O motor define dois papéis via LangChain: *rewriter* (GPT-4o, apoio à interpretação de dados brutos) e *analyst* (GPT-5, modelo de *reasoning* que elabora o parecer final). O *analyst* recebe um system prompt de arquiteto DevSecOps sênior e o grafo JSON como mensagem, e é envolvido por *with_structured_output(StrideReport)*: a resposta não é texto livre, é um objeto Pydantic validado — uma lista de riscos em que cada risco aponta o id exato do elemento afetado no grafo. Esse vínculo id → *bounding box* é o que sustenta a rastreabilidade visual (risco → recorte do diagrama). Como a chamada leva minutos, ela roda numa *thread* separada enquanto a barra de progresso anima contra o ETA histórico.

2.5. Fase 5: Interface do usuário e orquestração

O *framework* Streamlit orquestra o pipeline e renderiza o parecer, além de desenhar os overlays: caixas coloridas por classe, linhas tracejadas de *proximity_hints*, mapa numerado de riscos e o recorte destacado (600×400) de cada elemento afetado, também consolida os riscos por elemento arquitetural (*group_risks*): um API Gateway com 4 ameaças STRIDE vira um bloco com uma imagem e uma tabela de 4 linhas, ordenado pela pior severidade e gera o PDF com ReportLab a partir dos mesmos dados da tela (sem recálculo), em tema claro/editorial.

3. RESULTADOS E DISCUSSÃO

3.1.1. UI do *framework* personalizada com a identidade visual da FIAP

A interface confeccionada reaproveitou a estilização da FIAP Pós-Tech, com o destaque para o *framework* STRIDE-AI, além de uma breve descrição do funcionamento de *back-end* da ferramenta, assim como, o botão de *upload* do diagrama de arquitetura, veja na figura 1.

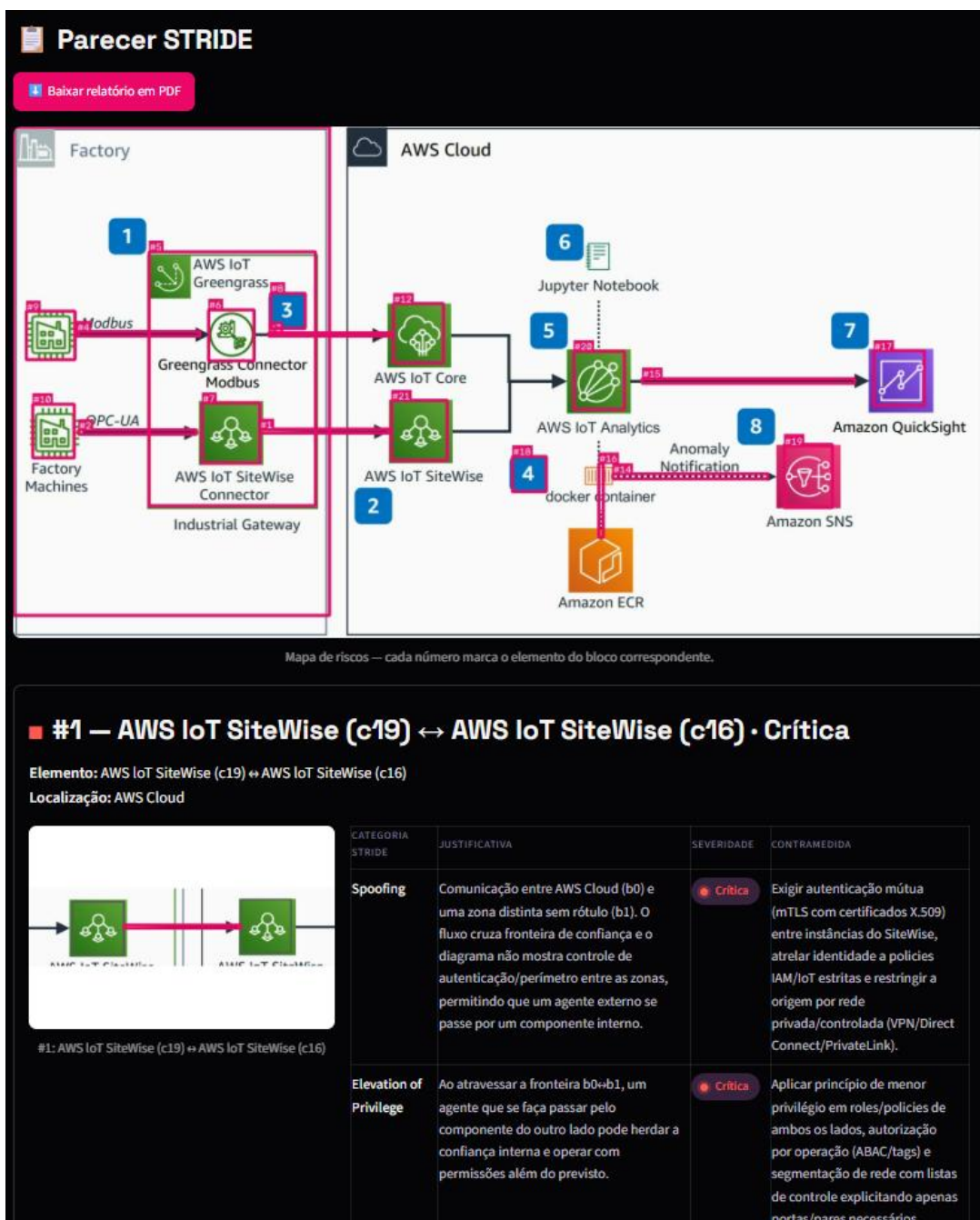
Figura 1 - *User interface* do STRIDE-AI com identidade visual inspirada na FIAP



Fonte: Autoria própria, 2026

Logo, após o envio de uma imagem de diagrama arquitetural, os *bounding boxes* são desenhados segundo cada classe definida, dessa forma, o resultado é exibido integralmente, ademais disso, aplica-se um *zoom* nas áreas e elementos em que houve a identificação de ameaças, inerentes à metodologia STRIDE – categoria, justificativa, criticidade e contramedida – conforme ilustra a figura 2 abaixo.

Figura 2 - Parecer técnico STRIDE resultante do envio de um diagrama de arquitetura AWS



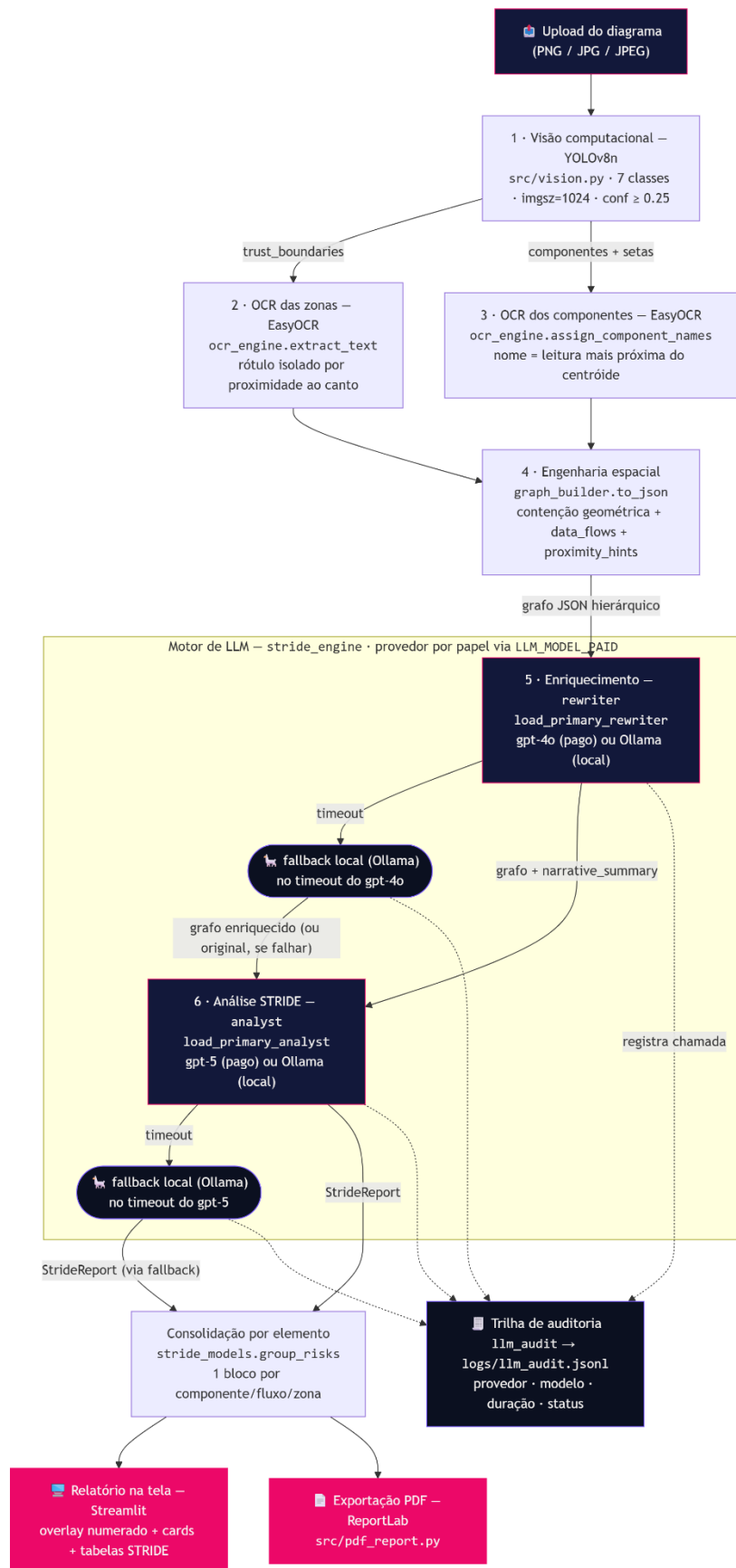
A arquitetura contém múltiplas tecnologias embarcadas em cada módulo, que estão enumeradas na tabela 1. O pipeline é síncrono e encadeado em 5 etapas, ao clicar em "Analisar diagrama" a imagem percorre visão computacional, OCR, engenharia espacial, grafo, LLM, e o parecer é renderizado, conforme ilustra a figura 3.

Tabela 1 - Estratificação da designação das tecnologias por atribuição e módulo

Tecnologia	Papel na solução	Onde
YOLOv8 Nano (Ultralytics + PyTorch)	Detecção supervisionada dos componentes do diagrama (7 classes), <i>fine-tuned</i> em <i>dataset</i> próprio	train.py, src/vision.py, models/best.pt
EasyOCR (+ OpenCV)	Leitura dos rótulos reais do diagrama: nome das zonas de confiança e dos serviços	src/ocr_engine.py
LangChain (ecossistema LangChain/LangGraph)	Orquestração dos papéis de LLM e saída estruturada (with_structured_output injeta o schema Pydantic na chamada)	src/stride_engine.py
LLM DevSecOps (GPT-5 analyst + GPT-4o rewriter, via OpenAI)	Parecer STRIDE: o system prompt codifica a expertise de um arquiteto de segurança sênior	src/prompts.py
Streamlit	Interface web: upload, progresso, relatório interativo	src/main.py
HTML + CSS estruturado	Marca "FIAP Software Security": CSS injetado mirando do Streamlit, hero/rodapé em HTML, pills de severidade, tabelas de risco estilizadas —	src/branding.py, src/theme.py, .streamlit/config.toml
NumPy / Pillow / OpenCV	Processamento de imagem: recortes, upscale, overlays, destaque de riscos	src/visual_report.py, src/ocr_engine.py

Fonte: Autoria própria, 2026

Figura 3 - Diagrama de pipeline do *framework* STRIDE-AI desenvolvido como MVP



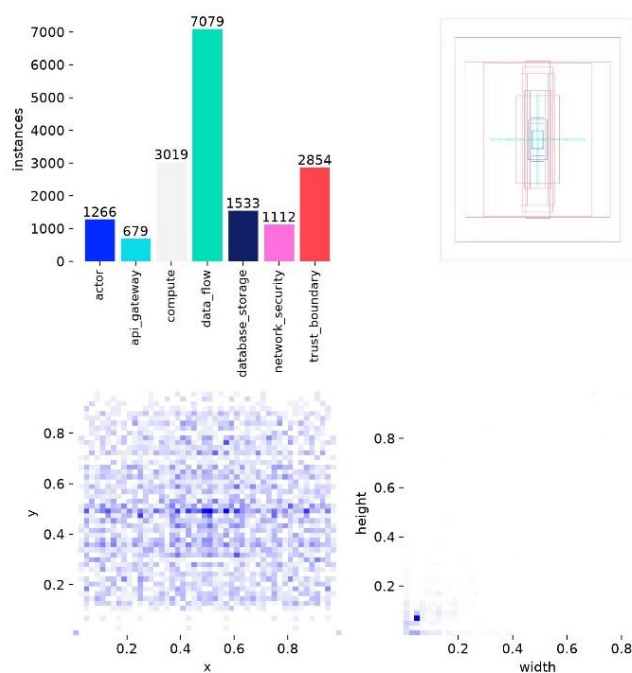
Fonte: Autoria própria, 2026

3.2. Métricas de treinamento do modelo YOLOv8

3.2.1. Rotulagem do *dataset*

A etapa de rotulagem de elementos por classe nas imagens de referência obtidas logrou êxito em inculir diversidade nas anotações. Todavia, naturalmente houve a sobreposição de classes mais comumente encontradas em detrimentos de outras mais incomuns, uma vez que a classe ‘api_gateway’ foi identificada 679 vezes; ‘actor’, ‘network_security’, ‘database_storage’ atingiram 1600 aparições; ‘compute’ e ‘trust_boundary’ próximos de 3000 e ‘data_flow’ com mais de 7000, segundo consta na figura 4.

Figura 4 - Distribuição de rótulos por classe e mapa de calor das regiões com mais classes



Fonte: Autoria própria, 2026

3.2.2. Desempenho por classes

No contexto de acuracidade média global da interpretação de arquitetura nos diagramas do conjunto de teste, observou-se precisão média de 60%, com notável desempenho de ‘api_gateway’ e ‘database_storage’ – ambas acima de 80% – tais constatações são denotadas pela tabela 2.

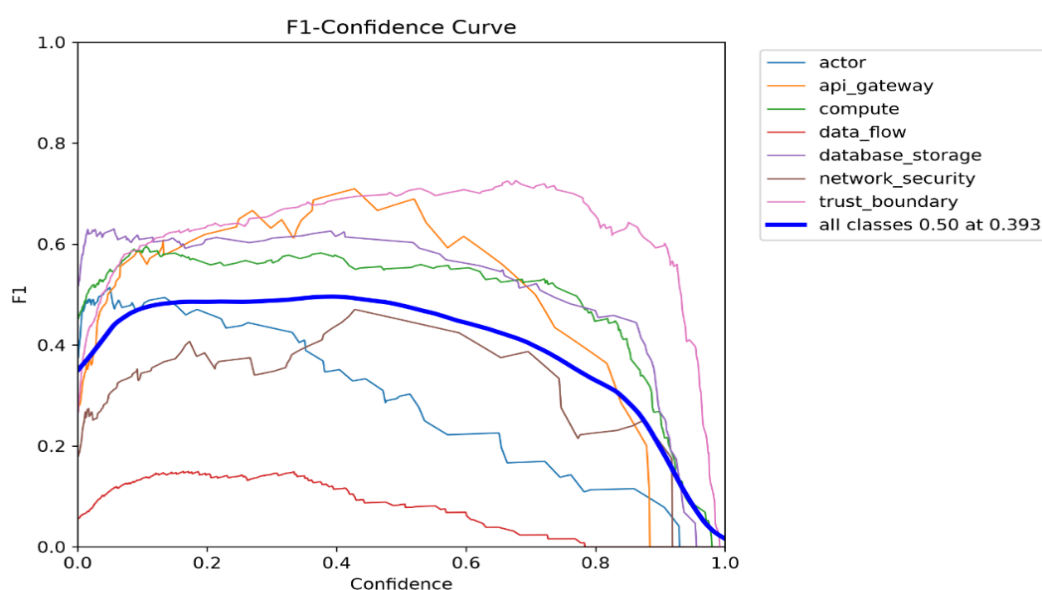
Tabela 2 - Performance média de acuracidade interpretativa das classes nos diagramas

Classe	Precisão	Recall	mAP50	mAP50-95
todas	0.603	0.444	0.496	0.239
api_gateway	0.814	0.611	0.742	0.280
trust_boundary	0.627	0.759	0.727	0.522
database_storage	0.872	0.485	0.624	0.257
compute	0.628	0.532	0.541	0.235
actor	0.510	0.265	0.404	0.152
network_security	0.503	0.381	0.376	0.210
data_flow	0.266	0.078	0.061	0.016

Fonte: Autoria própria, 2026

A partir da curva *F1-score* observa-se a proeminência das classes ‘trust_bondary’, ‘api_gateway’, ‘database_storage’ e ‘compute’ em comparação com as demais, pois estas possuem limiar de acurácia por *recall* >60%, enquanto as demais estão abaixo desse patamar, sendo este sobretudo influenciado de forma depreciativa pela classe ‘data_flow’, a qual teve precisão extremamente baixa, devido à dificuldade de diferenciação entre pixels de ruído e as linhas que apropriadamente descrevem o fluxo arquitetural, veja na figura 5.

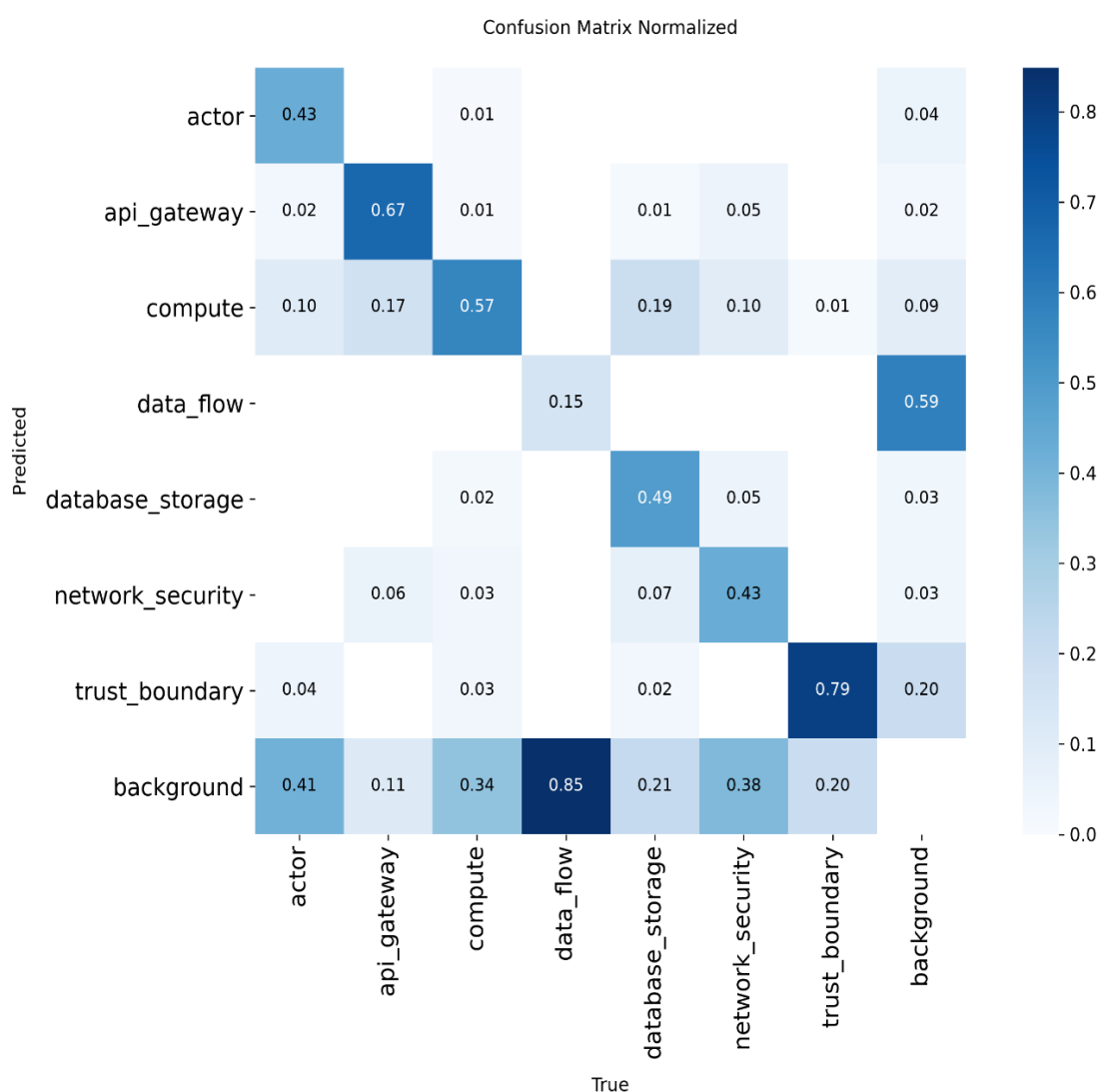
Figura 5 - Gráficos de progresso do aprendizado do modelo em face das épocas de treino



Fonte: Autoria própria, 2026

Logo, há mais confiabilidade do modelo em identificar as zonas de confiança, gerenciadores de rotas, banco de dados e servidores, contudo, as classes 'actor' e 'network_security' performaram com acurácia relativamente mais baixa (< 45%), sendo a classe mais crítica a 'data_flow', que apresentou apenas 15%, ademais disso, as taxas de acurácia no contexto exclusivo das *bounding boxes* globais – isto é, desconsiderando a taxa de precisão por diagrama testado – são apresentadas na figura 6.

Figura 6 - Matriz de confusão normalizada das *bounding boxes* globais por classe

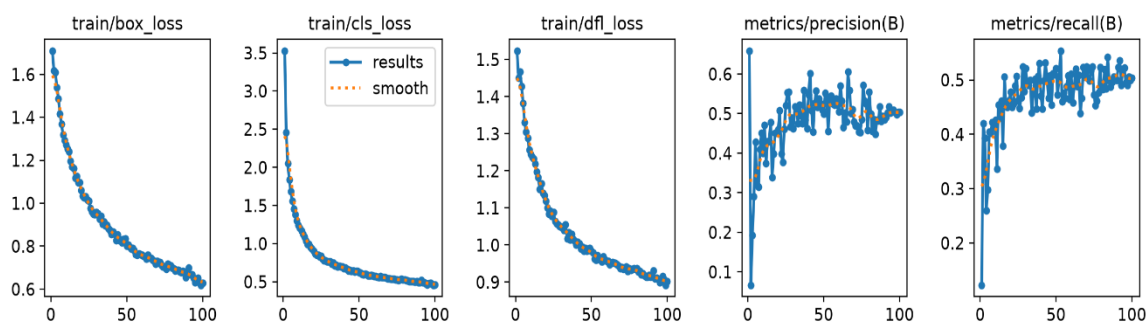


Fonte: Autoria própria, 2026

3.2.3. Desempenho por época

O progresso da métrica *train/box_loss* apresenta uma queda contínua e saudável de aproximadamente 1.6 para próximo de 0.6, isso indica que o modelo está aprendendo a desenhar as caixas delimitadoras (*bounding boxes*) com precisão cada vez maior nos dados de treino. Já *train/cls_loss* mostra uma redução acentuada no início e continua caindo de forma constante (chegando a cerca de 0.5), então o modelo está aprendendo a classificar corretamente quais objetos estão dentro das caixas no conjunto de treino. Por conseguinte, *train/dfl_loss* também apresenta uma curva decrescente consistente, o que significa que o modelo está refinando os detalhes finos das bordas das caixas delimitadoras durante o treino, conforme indica a figura 7.

Figura 7 - Gráficos de progresso do aprendizado do modelo em face das épocas de treino



Fonte: Autoria própria, 2026

3.2.4. Metadados do modelo final

O modelo YOLOv8n (nano) com *fine-tuning* via *framework* Ultralytics YOLO v8.4.92, cujo valor de parâmetros treináveis é 3.012.213 (~3,0 M), com *batch size* de 16 e tamanho padronizado em 1024×1024 para o *corpus* visual processado.

4. CONCLUSÃO

Colocando-se em retrospectiva, é peremptório salientar que intermediar o processo de rotulagem via plataforma Roboflow foi crucial para o MVP, porque a ferramenta agregou ganho de produtividade no desenho das *bounding boxes*, além de possibilitar a aplicação do aumento de dados de 63 para 439 exemplos rotulados. A partir disso, o desenvolvimento da proposta de avaliação de ameaças em diagramas de arquitetura demonstrou resultados promissores, contudo, ainda passíveis de melhorias, sobretudo em aumentar a diversidade e a robustez do *dataset*, pois 63 imagens-base não são suficientes para fomentar a capacidade de generalização almejada do *framework*. Por conseguinte, o estabelecimento da taxonomia canônica para diagramas *multicloud* justificou-se mediante à pluralidade de ícones e serviços correspondentes para cada classe, de tal forma que usar uma cardinalidade maior prejudicaria o aprendizado do modelo pela baixa quantidade de exemplos, não obstante, a classe ‘*data_flow*’ obteve o menor índice de acuracidade por haver variações de estilos da espessura, direção e comprimento das linhas e setas, desta forma, o modelo apresentou dificuldade de capturar todos os padrões inerentes a este elemento. Além disso, a escolha pelo modelo YOLOv8 nano provou-se razoável, tendo em vista o trinômio do custo-benefício: complexidade, agilidade e acuracidade; pois o *fine-tuning* levou cerca de 6h e apresentou um prognóstico animador. Outro fator relevante é a estrutura JSON definida no grafo, pois trouxe rastreabilidade e relacionamento entre os objetos identificados pelo modelo, logo, tais contribuições enriqueceram a análise STRIDE a posteriori.

Consoante a isso, a camada de OCR embarcada tornou-se um fator-chave na interpretação do tipo de serviço específico atrelado a cada objeto rastreado, porque associou as nomenclaturas oficiais dos produtos das plataformas aos objetos identificados, assim como o *fallback* criado para contornar o problema de baixa detecção de linhas do fluxo (‘*data_flow*’) preservou relações arquiteturais. Então aproveitando-se dessa estrutura, o motor de modelagem STRIDE para geração das contramedidas municiado com LLM desempenha dupla responsabilidade, as quais são: *rewriter* para interpretação dos metadados enriquecidos – via YOLOv8, OCR e relações espaciais – e *analyst* com enfoque na avaliação dos riscos intrínsecos na

arquitetura proposta pelo diagrama enviado. Para tanto, foi realizada uma pré-
etapa de *prompt engineering* para a confecção dos *prompts* adequados para
cada atribuição via LangChain, ademais disso, essa sistemática elevou o grau
de capacidade analítica e revelou-se crucial para o retorno de alto valor entregue
pela aplicação. Por fim, temos a camada de *front-end*, a qual integrou um
framework manutenível a longo prazo, o Streamlit, capaz de permitir razoável
nível de personalização de layout e design, além de suportar a integração de
objetos *python* com navegadores *web*.

Já os resultados de interpretabilidade dos diagramas no conjunto de teste
trouxeram as classes críticas, entre elas estão: 'data_flow', 'network_security' e
'actor'; da mais crítica para a menos crítica, respectivamente, pois ao considerar-
se a média de acuracidade apenas nas classes cuja confiança é superior a 51%,
temos aproximadamente 75% de precisão, ou seja, a baixa taxa de acuracidade
para as classes críticas é responsável pela redução de 15% da média global de
precisão de interpretação dos diagramas, que girou em torno de 60% no total.
Outrossim, ao contrário do que se acredita convencionalmente, apesar da classe
'data_flow' ter tido o maior número de rótulos para treino (mais de 7000), ainda
sim performou em pior patamar, para entender a situação a métrica de mAP50
traz uma evidência, de que as *bounding boxes* desta classe não seguem um
padrão claro de posicionamento, desta forma, alternando a aparição de linhas e
setas com alto teor de imprevisibilidade e dispersão geométrica.

Portanto, o estágio embrionário da proposta permite o experimento de
melhorias, em especial em elevar a quantidade de diagramas com elementos
rotulados, com enfoque nas classes de 'actor' e 'network_security' inicialmente,
pois estas tendem a atingirem maiores patamares de acuracidade com maior
facilidade, ao contrário de 'data_flow', logo, isso já impactaria na média global de
acuracidade do modelo, que possivelmente chegaria aos 75% globais. Não
obstante, é possível elaborar outra estratégia de captura do fluxo arquitetural e
das relações dos elementos, sabendo-se que o *fallback* não é suficiente, então
uma proposição é a de encontrar padrões de relacionamento mais comumente
presentes entre as classes e inferir o fluxo com maior assertividade, dessa forma,
a tendência é a indubitável evolução do *framework*.

5. REFERÊNCIAS BIBLIOGRÁFICAS

1. BRANCO, Marina. Hacker anuncia Neymar no site do Coritiba. Correio, Salvador, 4 jan. 2025. Disponível em: <https://www.correio24horas.com.br/esportes/hacker-anuncia-neymar-no-site-do-coritiba-0125>. Acesso em: 16 jul. 2026.
2. AMERISE, Atahualpa. Ashley Madison: o mega hackeamento que expôs dados de milhões de casados infiéis (e o que aconteceu com a empresa). BBC News Brasil, 18 maio 2024. Disponível em: <https://www.bbc.com/portuguese/articles/cgrr2k8glepo>. Acesso em: 16 jul. 2026.
3. VERCEL. Vercel April 2026 security incident. Vercel Knowledge Base, 2026. Última atualização: 24 abr. 2026. Disponível em: <https://vercel.com/kb/bulletin/vercel-april-2026-security-incident>. Acesso em: 16 jul. 2026.
4. REDAÇÃO IMASTERS. Nova IA brasileira ajuda desenvolvedores a proteger aplicações contra riscos de segurança. iMasters, 10 set. 2025. Disponível em: <https://imasters.com.br/noticia/nova-ia-brasileira-ajuda-desenvolvedores-a-proteger-aplicacoes-contr-riscos-de-seguranca>. Acesso em: 16 jul. 2026.
5. MARTINS, Gustavo Lima. Hackathon STRIDE-AI. GitHub, 2026a. Disponível em: <https://github.com/GustavoLimaMartins/hackathon-stride-ai>. Acesso em: 17 jul. 2026.
6. MARTINS, Gustavo Lima. Apresentação Hackathon FIAP – IA para Devs (Fase 5). YouTube, 2026b. Disponível em: https://youtu.be/5H0NWx_G0nE. Acesso em: 17 jul. 2026.

7. MICROSOFT. Azure Architecture Center. Microsoft Learn, [2026]. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/>. Acesso em: 16 jul. 2026.
8. AMAZON WEB SERVICES (AWS). AWS Reference Architecture Diagrams. AWS Architecture Center, [2026]. Disponível em: <https://aws.amazon.com/pt/architecture/reference-architecture-diagrams/>. Acesso em: 16 jul. 2026.
9. GOOGLE CLOUD. Cloud Architecture Center. Google Cloud Documentation, [2026]. Disponível em: <https://docs.cloud.google.com/architecture?hl=pt-br>. Acesso em: 16 jul. 2026.
10. ROBOFLOW. Roboflow Annotate: The Fastest Way to Label Computer Vision Datasets. Roboflow, [2026]. Disponível em: <https://roboflow.com/annotate>. Acesso em: 16 jul. 2026.
11. MARTINS, Gustavo Lima. Hackathon: dataset anotado para detecção de componentes de arquitetura de software. Roboflow, versão 12, 2026c. Disponível em: <https://app.roboflow.com/gustavo-lima-vprcy/hackathon-vp4zy/> 12. Acesso em: 16 jul. 2026.